# UNIT - IV: A)Java Collections B) JDBC C)I/O Streams D) Reflection

## A. Collection Framework

**1. Overview**
- Role and importance of Collection Framework
- Advantages over arrays

**2. Core Interfaces**
- **List**: ArrayList, LinkedList
- **Set**: HashSet, TreeSet, LinkedHashSet
- **Map**: HashMap, TreeMap, LinkedHashMap

**3. Iterators**
- Iterator and ListIterator
- Difference between **Enumeration** and **Iterator**

**4. Searching and Algorithms**
- Searching elements in a List
- Collections utility class (e.g., `Collections.sort()`, `binarySearch()`, `reverse()`, `shuffle()`)

**5. Hash vs Tree-based Collections**
- HashMap vs TreeMap
- HashSet vs TreeSet

**6. Object Equality**
- `equals()` and `hashCode()` methods and their contracts

**7. Comparisons**
- **Comparable** vs **Comparator**

**8. Thread Safety**
- Thread-safe collections: `Vector`, `Collections.synchronizedList()`
- Use of `ConcurrentHashMap` (optional advanced topic)

**9. Generics**
- Type safety
- Generic methods and classes

**10. Properties Class**
- Using `Properties` to manage configuration via `.properties` files

# Collection Framework in Java

## 1. OVERVIEW: Role and importance of Collection Framework

### What is the Collection Framework?

The Collection Framework is a unified architecture to store, retrieve, and manipulate a group of objects efficiently.

### Core Interfaces and Classes

| Interface | Implementing Classes | Description |
|---|---|---|
| **List** | ArrayList, LinkedList, Vector | Ordered collection, allows duplicates |
| **Set** | HashSet, TreeSet, LinkedHashSet | No duplicates allowed |
| **Map** | HashMap, TreeMap, LinkedHashMap | Stores key-value pairs |
| **Queue** | PriorityQueue, LinkedList | Follows FIFO order |

### Role Of Collection Framework

The Collection Framework in Java provides ready-made implementations like ArrayList, HashMap, HashSet, etc., which:

- Save development time
- Are tested and optimized
- Improve code readability and maintainability

### Importance of Collection Frame work

1. **Standardized Architecture**
   Provides a uniform interface for various data structures, making the code easier to understand and use.
2. **Ready-to-Use Data Structures**
   Includes popular implementations: ArrayList, LinkedList, HashSet, TreeSet, HashMap, etc.
3. **Built-in Algorithms**
   The Collections class includes sorting, searching, shuffling, etc., so we don't have to write these from scratch.
4. **Interoperability**
   Different collection classes can be easily converted or used together (e.g., List to Set).
5. **Supports Generics**
   Ensures type safety, so runtime errors are reduced.
6. **Thread-Safe Options Available**
   Provides thread-safe classes like Vector and wrappers using Collections.synchronizedList().
7. **Flexible Iteration**
   With Iterator and ListIterator, we can easily traverse and even modify collections during iteration.

### Advantages of Collection Framework over Arrays

| Feature | Arrays | Collection Framework |
|---|---|---|
| **Size** | Fixed size | Dynamic size (auto-grow/shrink) |
| **Data Types** | Can store primitives | Stores objects (Generics for type safety) |
| **Operations** | Manual (sort, search) | Built-in algorithms in `Collections` class |
| **Flexibility** | Only index-based | Supports various data structures (List, Set, Map) |
| **Performance Tuning** | Limited | Multiple implementations to choose based on need |
| **Thread-Safety Options** | No | Yes, via synchronized collections |
| **Type Safety** | No by default | Yes, using Generics |

## 2. Core Interfaces in the Collection Framework[ LIST, SET, MAP ]

### 1. List Interface

- Ordered collection (maintains insertion order)
- Allows duplicate elements
- Access elements by index

◆ **Common Implementations:**

| Class | Description |
|---|---|
| ArrayList | Resizable array, fast for reading/accessing, slower for insert/delete |
| LinkedList | Doubly linked list, better for frequent insertions/deletions |

**Example:**

```
List<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");
names.add("Alice"); // allows duplicate
System.out.println(names); // [Alice, Bob, Alice]
```

### 2. Set Interface

- Unordered collection (except LinkedHashSet)
- **Does not allow duplicates**

◆ **Common Implementations:**

| Class | Description |
|---|---|
| HashSet | Unordered, fast access using hash table |
| TreeSet | Sorted (natural or custom order), slower |
| LinkedHashSet | Maintains **insertion order** |

◆ **Example:**

```
Set<Integer> numbers = new HashSet<>();
numbers.add(10);
numbers.add(20);
numbers.add(10); // duplicate, ignored
System.out.println(numbers); // [10, 20]
```

### 3. Map Interface

- Stores key-value pairs
- Keys must be unique
- Values can be duplicated

◆ **Common Implementations:**

| Class | Description |
|---|---|
| **HashMap** | **Unordered, fast access** |
| **TreeMap** | **Sorted by key (natural or custom comparator)** |
| **LinkedHashMap** | **Maintains insertion order of keys** |

**Examlple: :**

```
Map<String, Integer> scores = new HashMap<>();
scores.put("Alice", 90);
scores.put("Bob", 85);
scores.put("Alice", 95); // replaces old value
System.out.println(scores); // {Alice=95, Bob=85}
```

# 3. Iterators in Java

- Iterators are used to traverse collections like List, Set, etc., without using index-based access.

**1. Iterator Interface**

- Can be used with any Collection (List, Set, etc.)
- Forward-only traversal
- Allows safe removal of elements while iterating

◆ **Methods:**

- boolean hasNext()
- E next()
- void remove()

**Example: :**

```
List<String> names = List.of("Alice", "Bob", "Charlie");
Iterator<String> itr = names.iterator();

while (itr.hasNext())
{
   System.out.println(itr.next());
}
```

**2. ListIterator Interface**

Used only with List implementations (e.g., ArrayList, LinkedList)
Bi-directional traversal (forward and backward)
Allows element modification

◆ **Extra Methods (in addition to Iterator):**

- boolean hasPrevious()
- E previous()
- void set(E e)
- void add(E e)

◆ **Example:**

```
List<String> colors = new ArrayList<>(List.of("Red", "Green", "Blue"));
ListIterator<String> listItr = colors.listIterator();

while (listItr.hasNext())
{
   System.out.println("Forward: " + listItr.next());
}

while (listItr.hasPrevious())
{
   System.out.println("Backward: " + listItr.previous());
}
```

## 3.Difference between Enumeration and Iterator

| Feature | Enumeration | Iterator |
|---|---|---|
| **Origin** | Legacy (used with Vector, Hashtable) | Modern (all collections) |
| **Direction** | Only forward | Only forward |
| **Methods** | hasMoreElements(), nextElement() | hasNext(), next(), remove() |
| **Remove support** | Not supported | remove() supported |
| **Fail-fast behavior** | No | Yes (ConcurrentModificationException) |

◆

Example of Enumeration:

```
Vector<String> v = new Vector<>(List.of("A", "B", "C"));
Enumeration<String> e = v.elements();

while (e.hasMoreElements())

{
    System.out.println(e.nextElement());
}
```

## 4. Searching and Algorithms in Java Collections- java.util.Collections.

The java.util.Collections class provides **static utility methods** to perform common operations like searching, sorting, shuffling, and reversing on collection objects like List.

### 1. Searching Elements in a List

- **Manual Search (Linear Search)**

```
List<String> names = List.of("Alice", "Bob", "Charlie");
if (names.contains("Bob"))
{
   System.out.println("Found Bob!");
}
```

- **Binary Search :**

The list **must be sorted** before using binarySearch().

```
List<Integer> numbers = new ArrayList<>(List.of(10, 20, 30, 40, 50));
int index = Collections.binarySearch(numbers, 30);
System.out.println("Found at index: " + index);  // Output: 2
```

### 2. Collections Utility Methods

**a) Collections.sort(List) :**

Sorts the list in **natural order** (ascending for numbers, alphabetical for strings).

```
List<Integer> nums = new ArrayList<>(List.of(4, 2, 9, 1));
Collections.sort(nums);
System.out.println(nums);  // [1, 2, 4, 9]
```

You can also pass a custom Comparator for custom sorting.

**b) Collections.reverse(List)**

Reverses the order of elements in the list.

```
Collections.reverse(nums);
System.out.println(nums);  // [9, 4, 2, 1]
```

**c) Collections.shuffle(List)**

Randomly reorders elements in the list — useful for games or testing.

```
Collections.shuffle(nums);
System.out.println(nums);  // Output varies each time
```

**d) Collections.binarySearch(List, key)**

Searches for the given key using binary search — list must be sorted first.

```
int pos = Collections.binarySearch(nums, 4);
System.out.println("Index of 4: " + pos);
```

## 5. Hash vs Tree-based Collections in Java

Java provides **two major types of collection implementations** based on internal data structure:

- **Hash-based** (uses **hashing**)
- **Tree-based** (uses **binary search trees**, typically **Red-Black Trees**)

### 1. HashMap vs TreeMap

| Feature | HashMap | TreeMap |
|---|---|---|
| **Ordering** | No ordering (random) | Sorted order (based on keys) |
| **Null keys** | Allows 1 null key | Null keys not allowed |
| **Performance** | Faster: O(1) for get/put | Slower: O(log n) for get/put |
| **Internal Structure** | Hash table | Red-Black Tree |
| **Use Case** | When order doesn't matter | When sorted keys are required |

```
Map<String, Integer> hashMap = new HashMap<>();
hashMap.put("Banana", 1);
hashMap.put("Apple", 2);
System.out.println(hashMap);  // Random order

Map<String, Integer> treeMap = new TreeMap<>();
treeMap.put("Banana", 1);
treeMap.put("Apple", 2);
System.out.println(treeMap);  // Sorted: {Apple=2, Banana=1}
```

**When to Use What?**

- Use **HashMap/HashSet** when:
  - You want **fast access**
  - You **don't care about order**
- Use **TreeMap/TreeSet** when:
  - You need **sorted data**
  - You perform frequent **range**

### 2. HashSet vs TreeSet

| Feature | HashSet | TreeSet |
|---|---|---|
| **Ordering** | Unordered | Sorted (natural order or comparator) |
| **Null elements** | Allows 1 null element | Throws NullPointerException if comparator is used |
| **Performance** | Faster: O(1) for add/search | Slower: O(log n) |
| **Internal Structure** | Hash table | Red-Black Tree |
| **Duplicates** | Not allowed | Not allowed |
| **Use Case** | Quick existence check | Sorted set operations |

Example:

```
Set<String> hashSet = new HashSet<>();
hashSet.add("C");
hashSet.add("A");
hashSet.add("B");
System.out.println(hashSet);  // Random order

Set<String> treeSet = new TreeSet<>();
treeSet.add("C");
treeSet.add("A");
treeSet.add("B");
System.out.println(treeSet);  // Sorted: [A, B, C]
```

## 6. Object Equality in Java

In Java, **object equality** is determined using:

- == → checks **reference equality** (are both pointing to the same object)
- equals() → checks **logical equality** (are both objects *meaningfully* equal)
- hashCode() → used in **hash-based collections** to place and find objects efficiently

---

**1. equals() Method**

Used to compare the **contents or logical equality** of two objects.

- **Default Behavior:**

The default implementation (from Object class) uses ==, i.e., reference equality.

- **Override Example:**

```
public class Student
 {
        int id;
        String name;

        public Student (int id, String name)
            {
                this.id = id;
                this.name = name;
            }

        @Override
        public boolean equals (Object o)
            {
                if (this == o) return true;
                if (!(o instanceof Student)) return false;
                Student s = (Student) o;
                return this.id == s.id && this.name.equals(s.name);
            }
}
```

**2. hashCode() Method**

Returns an **int hash value** that determines the bucket where the object is stored in hash-based collections.

- Must be overridden whenever equals() is overridden.
- Same objects (equal ones) must return the same hash code.

```
@Override
public int hashCode()
{
   return Objects.hash(id, name);
}
```

## equals() and hashCode() Contract

| Rule | Description |
|---|---|
| 1. **Consistent** | Repeated calls to equals() and hashCode() must return the same result, unless object changes. |
| 2. **If equals() is true → hashCode() must be equal** | Two equal objects must have the same hash code. |
| 3. **If hashCode() is equal → equals() can be true or false** | But for best performance in hash collections, equal hashCodes should mean equals() is likely true. |

**What Happens If You Violate the Contract?**
If you override equals() but not hashCode(), or vice versa:

- Hash-based collections like HashMap, HashSet, and Hashtable **won't work correctly**.
- You may get **duplicates**, **unable to find objects**, or inconsistent behavior.

Example with HashSet:

```
Set<Student> set = new HashSet<>();
set.add(new Student(1, "Alice"));
set.add(new Student(1, "Alice")); // duplicate logically

System.out.println(set.size()); // 2 if equals/hashCode not properly overridden, 1 if correct
```

## 7. Comparisons

**1. Comparable Interface**

- Belongs to: java.lang
- Used for: **Natural ordering** (default sorting)
- Implemented **inside the class** whose objects need to be compared

◇ **Method:**

int compareTo(T o)

◇ Example:

```
public class Student implements Comparable<Student> {
   int id;
   String name;

   public Student(int id, String name) {
      this.id = id;
      this.name = name;
   }

   @Override
   public int compareTo(Student other) {
      return this.id - other.id; // Ascending order by id
   }
}
```

```
List<Student> list = new ArrayList<>();
Collections.sort(list);  // Uses compareTo()
```

**2. Comparator Interface**

**Belongs to: java.util**

**Used for: Custom ordering**

Implemented outside the class, often using anonymous classes or lambdas

◇ **Method:**

int compare(T o1, T o2)

◇ **Example:**

```
class NameComparator implements
Comparator<Student> {
    public int compare(Student s1, Student s2) {
        return s1.name.compareTo(s2.name);  //
Alphabetical order
    }
}
```

**Collections.sort(list, new NameComparator()); // Uses compare()**

**When to Use What?**

| Situation | Use |
|---|---|
| You want one fixed default sorting order | Comparable |
| You want multiple ways to sort (by name, age, etc.) | Comparator |

**Bonus: Java 8 Lambda Example for Comparator**

```
Collections.sort(list, (s1, s2) -> s1.name.compareTo(s2.name));
```

## 8. Thread Safety in Java Collections

**What is Thread Safety?**
A thread-safe collection ensures that it behaves correctly when multiple threads access it simultaneously, especially when modifications (add, remove, update) are involved.
If a collection is not thread-safe, using it in a multi-threaded program can lead to unexpected behaviour or data corruption.

**1. Vector (Legacy Thread-Safe List)**

- Vector is part of the legacy Java collection classes.
- All methods are synchronized — only one thread can access a method at a time.
- Ensures thread safety but can be slow due to unnecessary synchronization.

**Example:**

```
import java.util.Vector;
public class VectorExample {
   public static void main(String[] args) {
      Vector<Integer> v = new Vector<>();
      v.add(10);
      v.add(20);
      System.out.println(v);
   }
}
```

**2. Collections.synchronizedList()**

- **Wraps any non-thread-safe List (like ArrayList) into a thread-safe version.**
- **Provides flexibility over using Vector.**
- **Still requires manual synchronization during iteration.**

**Example:**

```
import java.util.*;

public class SyncListExample {
   public static void main(String[] args) {
      List<String> list = Collections.synchronizedList(new ArrayList<>());
      list.add("Apple");
      list.add("Banana");

      // Manual sync during iteration
      synchronized (list) {
         for (String fruit : list) {
            System.out.println(fruit);
         }
      }
   }
}
```

**3. ConcurrentHashMap ( Optional Advanced Topic)**

- **A modern thread-safe Map for high-concurrency scenarios.**
- **Allows multiple threads to read and write concurrently without locking the whole map.**
- **Much faster than Hashtable or synchronized maps.**

**Example:**

```
import java.util.concurrent.ConcurrentHashMap;
public class ConcurrentMapExample
{
   public static void main(String[] args)
{
 ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();
       map.put("A", 100);
       map.put("B", 200);

       System.out.println(map.get("A"));
   }
}
```

- Key Features:
  - Efficient in multi-threaded environments
  - No need to manually synchronize
  - Used heavily in real-time systems, web servers, etc.

# 9. Generics in Java

**Generics allow you to write type-safe code by specifying a data type as a parameter, like List<String> or Map<Integer, String>. Introduced in Java 5 to avoid runtime errors caused by improper type casting.**

## 1. Type Safety with Generics

Without generics, collections store Objects, so you must manually cast them when retrieving elements — which can cause runtime errors.

- **Without Generics:**

```
List list = new ArrayList();  // raw type
list.add("Hello");
list.add(100);  // Compiles fine, but risky

String str = (String) list.get(0);  // OK
String num = (String) list.get(1);  //Runtime error (ClassCastException)
```

- **With Generics:**

```
List<String> list = new ArrayList<>();
list.add("Hello");
// list.add(100);  // Compile-time error

String str = list.get(0);  // No casting needed
```

**Type Safety ensures that only the correct data types are used.**

## 2. Generic Classes

You can define a class with a type parameter. This makes it reusable for different data types.

- **Example:**

```
public class Box<T> {
   private T value;

   public void set(T value) {
      this.value = value;
   }

   public T get() {
      return value;
   }
}
```

- **Usage:**

```
Box<String> stringBox = new Box<>();
stringBox.set("Hello");
System.out.println(stringBox.get());  // Output: Hello

Box<Integer> intBox = new Box<>();
intBox.set(100);
System.out.println(intBox.get());  // Output: 100
```

**3. Generic Methods**

**You can also define methods that work with any type, without making the entire class generic.**

- **Example:**

```
public class GenericUtils
{
   public static <T> void printArray(T[] array)
   {
      for (T item : array)
      {
         System.out.println(item);
      }
   }
}
```

- **Usage:**

```
String[] names = {"Alice", "Bob"};
Integer[] numbers = {1, 2, 3};

GenericUtils.printArray(names);    // Works with String[]

GenericUtils.printArray(numbers);  // Works with Integer[]
```

# 10. Using the Properties Class in Java

**What is the Properties Class?**

- A subclass of Hashtable<Object, Object> from java.util.
- Used to store key-value pairs as Strings.
- Commonly used to load/save application configurations from/to .properties files.

**1. Creating and Storing a .properties File**

You can write configuration data (like database settings, API keys, etc.) into a .properties file:

◇ config.properties file example:

```
username=admin
password=12345
dburl=jdbc:mysql://localhost:3306/mydb
```

**2. Loading a Properties File in Java**

**Example:**

```
import java.util.Properties;
import java.io.FileReader;
import java.io.IOException;

public class LoadConfig {
   public static void main(String[] args)
  {
      Properties props = new Properties();
      try {
         FileReader reader = new FileReader("config.properties");
         props.load(reader);
         String username = props.getProperty("username");
         String password = props.getProperty("password");
         System.out.println("Username: " + username);
         System.out.println("Password: " + password);
      }
       catch (IOException e) {    e.printStackTrace();
      }
   }
}
```

**3. Writing (Storing) Properties to a File**

**Example:**

```
import java.util.Properties;
import java.io.FileWriter;
import java.io.IOException;

public class SaveConfig {
   public static void main(String[] args) {
      Properties props = new Properties();
      props.setProperty("app.name", "MyApp");
      props.setProperty("version", "1.0");
      props.setProperty("maintainer", "admin@example.com");
      try {   FileWriter writer = new FileWriter("app-config.properties");
         props.store(writer, "App Configuration");
         System.out.println("Properties saved successfully.");
      } catch (IOException e) {
         e.printStackTrace();
      }
   }
}
```

**The second argument in store() is a comment added at the top of the .properties file.**

**Key Methods of Properties:**

| Method | Description |
|---|---|
| load(InputStream) | Loads properties from a file |
| store(OutputStream, String) | Saves properties to a file |
| getProperty(String key) | Gets the value of a key |
| setProperty(String key, String value) | Sets a key-value pair |

**Real-World Uses of Properties Files:**

- Database connection configs
- Application settings (themes, language)
- API keys and tokens
- Logging configuration

## JDBC (Java Database Connectivity)

### 1. Driver Types

**Native and ODBC Drivers**

- **ODBC (Open Database Connectivity)**: Generic interface for different databases.
- **Native Drivers**: Specific to the database (e.g., Oracle, MySQL drivers).

◆ **Types of JDBC Drivers**

| Type | Name | Description |
|---|---|---|
| **Type 1** | JDBC-ODBC Bridge | Maps JDBC calls to ODBC. Deprecated and removed after Java 8. |
| **Type 2** | Native-API Driver | Converts JDBC calls to native DB calls using platform-specific libraries. |
| **Type 3** | Network Protocol Driver | Uses middleware to connect to DB. Useful in multi-tier applications. |
| **Type 4** | Thin Driver (Pure Java) | Directly converts JDBC to DB protocol. Most widely used driver today. |

## 2. Setting up a Connection

- **Load Driver and Connect**

```
Class.forName("com.mysql.cj.jdbc.Driver");
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/mydb", "user", "password");
```

- **Using Properties for Connection Factory**

```
Properties props = new Properties();
props.setProperty("user", "admin");
props.setProperty("password", "1234");

Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/mydb", props);
```

## 3. Basic CRUD Operations

- **INSERT**

```
Statement stmt = conn.createStatement();
stmt.executeUpdate("INSERT INTO users (name, age) VALUES ('Neha', 36)");
```

- **DELETE**

```
stmt.executeUpdate("DELETE FROM users WHERE id = 1");
```

- **UPDATE**

```
stmt.executeUpdate("UPDATE users SET age = 35 WHERE name = 'Neha'");
```

- **SELECT**

```
ResultSet rs = stmt.executeQuery("SELECT * FROM users");
while (rs.next()) {
   System.out.println(rs.getString("name") + " - " + rs.getInt("age"));
}
```

## 4. Statements in JDBC

- **A. Statement**
  - Executes static SQL queries.
  - Prone to SQL injection.
- **B. PreparedStatement (Safe & Efficient)**
  - Precompiled SQL template.
  - Prevents SQL injection.

```
PreparedStatement ps = conn.prepareStatement("INSERT INTO users (name, age) VALUES (?, ?)");
        ps.setString(1, "Bob");
        ps.setInt(2, 30);
        ps.executeUpdate();
```

- **C. CallableStatement (Stored Procedures/PLSQL)**

```
        CallableStatement cs = conn.prepareCall("{call addUser(?, ?)}");
        cs.setString(1, "Tom");
        cs.setInt(2, 28);
        cs.execute();
```

**Batch Updates**

```
PreparedStatement ps = conn.prepareStatement("INSERT INTO log (msg) VALUES (?)");
ps.setString(1, "Log 1"); ps.addBatch();
ps.setString(1, "Log 2"); ps.addBatch();
ps.executeBatch();
```

---

- **Transactions**

```
conn.setAutoCommit(false);
stmt.executeUpdate("INSERT INTO ...");
stmt.executeUpdate("UPDATE ...");
conn.commit();  // or conn.rollback();
```

---

## 5. Metadata

- **ResultSetMetaData**

Gives info about the columns in a result set.

```
ResultSetMetaData meta = rs.getMetaData();
int cols = meta.getColumnCount();
for (int i = 1; i <= cols; i++) {
    System.out.println(meta.getColumnName(i));
}
```

- **DatabaseMetaData**

Gives info about the database itself.

```
DatabaseMetaData dbmd = conn.getMetaData();
System.out.println("Product: " + dbmd.getDatabaseProductName());
```

## 6. Database Initialization

- **Create tables programmatically**

```
Statement stmt = conn.createStatement();
    stmt.executeUpdate("CREATE TABLE users (id INT PRIMARY KEY AUTO_INCREMENT, name
    VARCHAR(50), age INT)");
```

**Input/Output (I/O) Streams in Java**

In Java, I/O streams are used to read from and write to different data sources such as files, network connections, and memory.

## ✅ 1. Stream Types

**Java has two main types of streams:**

### A. Byte Streams

- Handles binary data (images, audio, PDF, etc.)
- Classes:
  - InputStream – abstract class for reading bytes
  - OutputStream – abstract class for writing bytes

**Example:**

```
FileInputStream fis = new FileInputStream("input.txt");
int data;
while ((data = fis.read()) != -1)
{
   System.out.print((char) data);
}
fis.close();
```

### B. Character Streams

- Handles text data (Unicode characters)
- Classes:
  - Reader – abstract class for reading characters
  - Writer – abstract class for writing characters

**Example:**

```
FileReader fr = new FileReader("input.txt");
int ch;
while ((ch = fr.read()) != -1)
 {
   System.out.print((char) ch);
}
fr.close();
```

## ✅ 2. Filters and Buffers

---

### 🔹 A. Filter Streams

- Used to add additional functionality (like reading only part of a stream)
- **Subclasses:**
  - FilterInputStream
  - FilterOutputStream

Used as base classes for Buffered, Data, and Pushback streams.

---

### 🔹 B. Buffered Streams

- Improve performance by reducing disk read/write operations.

#### 🔸 BufferedInputStream:

```
BufferedInputStream bis = new BufferedInputStream(new FileInputStream("input.txt"));
int b;
while ((b = bis.read()) != -1) {
   System.out.print((char) b);
}
bis.close();
```

#### 🔸 BufferedOutputStream:

```
BufferedOutputStream bos = new BufferedOutputStream(new FileOutputStream("output.txt"));
bos.write("Hello Buffered Output!".getBytes());
bos.close();
```

## 3. Special Streams

### A. DataInputStream / DataOutputStream

- Used to read and write primitive data types (int, float, etc.)
- Works with binary data

- **Example:**

```
DataOutputStream dos = new DataOutputStream(new FileOutputStream("data.bin"));
dos.writeInt(100);
dos.writeFloat(99.5f);
dos.close();

DataInputStream dis = new DataInputStream(new FileInputStream("data.bin"));
int x = dis.readInt();
float y = dis.readFloat();
System.out.println(x + " " + y);
dis.close();
```

### B. PrintStream

- Used to print text to console or file with methods like print() and println().
- Example: System.out is a PrintStream

- **File Output Example:**

```
PrintStream ps = new PrintStream("output.txt");
ps.println("Hello from PrintStream!");
ps.close();
```

**C. RandomAccessFile**

- **Allows read and write to a file at any position.**
- **Useful for large files or databases.**

- **Example:**

```
RandomAccessFile raf = new RandomAccessFile("data.txt", "rw");
raf.writeUTF("Hello");
raf.seek(0);  // Move to beginning
System.out.println(raf.readUTF());
raf.close();
```

## D. Reflection API

**1. Basics**

- What is Reflection?
- Accessing fields, methods, constructors at runtime

**2. Methods**

- Class.forName()
- newInstance()
- Accessing private methods using setAccessible(true)

**3. Tools**

- javap (Java Class Disassembler)
- Creating your own javap-like tool (conceptual)

**4. Applet Viewer**

- Overview (Optional: Deprecated but historical context)

**5. Java 9 Features *(brief overview, not exhaustive)***

- JShell
- Modules
- Interface private methods

## Reflection API in Java

### 1. Basics

**What is Reflection?**

Reflection is a feature in Java that allows a program to **examine or modify its own structure** at runtime — including **classes**, **methods**, **constructors**, **fields**, etc.

It belongs to the java.lang.reflect package.

**Use Cases:**

- Debugging and testing tools (JUnit, Mockito)
- IDEs and frameworks (Spring, Hibernate)
- Serialization, dependency injection
- Accessing private members

**Accessing Class Members at Runtime**

```
Class<?> cls = Class.forName("java.lang.String");
Method[] methods = cls.getDeclaredMethods();
for (Method m : methods)
{
   System.out.println(m.getName());
}
```

## 2. Important Reflection Methods

- **Class.forName(String className)**
  - Loads a class at runtime using its name.

```
Class<?> c = Class.forName("java.util.ArrayList");
```

- **newInstance()**
  - Creates a new instance of a class dynamically.

```
Object obj = c.newInstance();  // Deprecated in newer versions; prefer `getDeclaredConstructor().newInstance()`
```

- **Accessing Private Members**

```
import java.lang.reflect.*;

class Secret {
    private void show() {
        System.out.println("Accessed private method!");
    }
}

public class ReflectPrivate {
    public static void main(String[] args) throws Exception {
        Secret s = new Secret();
        Method m = Secret.class.getDeclaredMethod("show");
        m.setAccessible(true);  // bypass access checks
        m.invoke(s);
    }
}
```

## 3. Tools

---

### 🔹 javap – Java Class Disassembler

**What is javap?**

javap is a command-line tool that comes with the JDK. It disassembles compiled .class files and shows the structure of the class — including its fields, methods, and bytecode (if requested).

Useful for debugging, understanding compiled code, or exploring standard Java classes.

---

### 🔧 Basic Syntax

```
javap ClassName
javap java.lang.String
javap -c ClassName        // Show bytecode
javap -public ClassName   // Show only public APIs
```

- Used to **inspect bytecode** of compiled .class files.

javap java.lang.String

**Example Output**

If you run:

```
javap java.util.ArrayList
```

You might see:

```
public class java.util.ArrayList<E> extends java.util.AbstractList<E> {
    public ArrayList();
    public boolean add(E);
    public E get(int);
    ...
}
```

- **Creating a Simple javap-like Tool (Conceptual)**
  - Use Reflection to print methods, fields, constructors of any class.

```
Class<?> c = Class.forName("java.lang.Math");
for (Method m : c.getDeclaredMethods()) {
    System.out.println(m);
}
```

  - Class.forName("java.lang.Math") dynamically loads the Math class using its **fully qualified name**.
  - The Class<?> type represents the class object for Math.
  - getDeclaredMethods() returns **all methods declared** in the Math class, including private, public, and static ones — **but not inherited ones**.
  - Each method is an object of the Method class from java.lang.reflect.
  - System.out.println(m) prints the **signature** of each method.

## 4. Applet Viewer (Optional – Historical)

**What is an Applet?**

- An **applet** is a small Java program that runs **inside a web browser** or **applet viewer**.
- It was typically used to create **interactive web-based applications** like animations, calculators, or games.
- Defined by extending java.applet.Applet or later javax.swing.JApplet.

**What is Applet Viewer?**

- appletviewer is a **command-line tool** provided by Java to **run and test applets without a browser**.
- It was especially useful for developers to debug applets during development.

**How It Worked:**

1. You write a .java class that extends Applet.
2. You write an .html file that embeds the applet using the <applet> tag.
3. You run it using:

appletviewer MyApplet.html

**Example Code: Hello Applet**

- **Java Class (HelloApplet.java)**

```
import java.applet.Applet;
import java.awt.Graphics;

public class HelloApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("Hello from Applet!", 20, 20);
    }
}
```

- **HTML File (HelloApplet.html)**

```
html
CopyEdit
<html>
  <body>
    <applet code="HelloApplet.class" width="300" height="100"></applet>
  </body>
</html>
```

---

**Deprecation Notice**

- As of **Java 9**, applets and appletviewer are **deprecated**.
- Completely removed in **Java 11** and beyond.
- Modern alternatives: **JavaFX**, **Swing**, or **web-based UI frameworks**.

---

## Java 9 Features (Brief Overview)

Java 9 introduced **major language and platform-level enhancements**. Below are the key features

---

### 1. JShell – The Java REPL (Read-Eval-Print Loop)

- A **command-line tool** to run Java statements **interactively** without writing a full class or method.
- Great for learning, prototyping, and testing code quickly.

**Example:**

```
$ jshell
jshell> int x = 10;
jshell> System.out.println(x * 2);
```

**Use Cases:**

- Teaching Java basics interactively
- Testing logic quickly
- Exploring APIs

---

### 2. Java Platform Module System (JPMS)

- Java 9 introduced a **modular system** for organizing code into modules using module-info.java.
- Helps in **encapsulation**, **dependency management**, and **performance optimization**.

**Syntax:**

```
// module-info.java
module com.myapp {
    requires java.sql;
    exports com.myapp.services;
}
```

**Use Cases:**

- Building large scalable applications
- Creating well-defined APIs
- Replacing complex classpath management

---

### 3. Private Methods in Interfaces

- Java 9 allows **private methods** inside **interfaces**.
- Useful for **code reusability** within default and static methods.

**Example:**

```
interface MyInterface {
    default void show() {
        log("Default show method");
    }

    private void log(String msg) {
        System.out.println("Log: " + msg);
    }
}
```

**Use Cases:**

- Cleaner and DRY interface code
- Hide internal logic within interfaces

---

# JDBC – Java Database Connectivity

**JDBC** → It is a Java API that allow Java Programs to connect to & interact with databases

**Database** → MySQL, Oracle, PostgreSQL etc

**Driver Manager** – Manages list of database drivers

**Connection** – Reprsent a connection to database

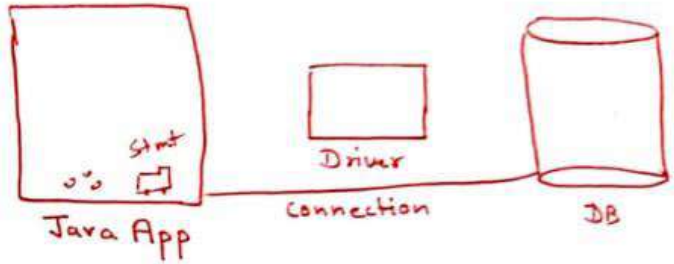
**Statement** – Use to execute static SQL queries.

**Prepared statement** – Used for parameterized SQL queries (prevent SQL injection)

**ResultSet** – Represent the result of a query. (like a table of data)

**Driver**
- → s/w component that enables Java App. to interact with a database.
- → Bridge b/w → Java program, → DBMS
- → Type – 1. JDBC-ODBC driver (depreciated)
  2. Native API driver
  3. N/w protocol driver
  4. Thin driver

## 1. Steps for DB Connection




download & add mysql-connector jar

1) load & Register Driver → class.forName(<driver path>);
2) Create connection → connection con = DriverManager.getConnection("url", "uname", "psw");
3) Create statement → PreparedStatement ps = con.prepareStatement("sql query");
4) Execute SQL statement → ps.executeQuery(); //select → ResultSet
   ~~prepared~~ ps.executeUpdate(); // ins, upd, del → int
5) Process the result
6) close the connection – con.close(), ps.close();

## ===== Steps to connect Java with Database using JDBC =====

1. Load & Register Driver
   => Class.forName("driver class");

2. Create connection
   => Connection con = DriverManager.getConnection("url", "username", "password");

3. Create statement
   => PreparedStatement ps = con.prepareStatement("sql query");

4. Execute SQL statement
   => ps.executeUpdate();

5. Process the result

6. Close the connection

# Simple prog. for JDBC

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class InsertDemo
{
    psvm(——)
    {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbc_db", "root", "root");
        PreparedStatement ps = con.prepareStatement("insert into register values('Neha', 'abc@gmail.com', '123', 'F', 'de
        int i = ps.executeUpdate();
        if (i > 0)
        {
            sop("Success");
        }
        else
        {
            sop("fail");
        }
        Con.close();
    }
}
```

mysql

```
> create database jdbc_db;
> use jdbc_db;
> create table register(name varchar(100), email varchar(100),
      gender varchar(10), city varchar(50));
> desc register;
> select * from register;
```

# JDBC Example [Update]

```
import java.sql.Connection;
import java.sql.DriverManager;

public class UpdateDemo
{
    psvm (-) throws Exception
    {
        String city1 = "pune"
        String email1 = "abc@gmail.com"

        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbc_db","root","root");
        PreparedStatement ps = con.prepareStatement("update register set city = ? where email = ?");
        ps.setString(1, city1);
        ps.setString(2, email1);
        int count = ps.executeUpdate();
        if (count > 0)
        {
            SOP("updated successfully");
        }
        else
        {
            SOP("FAIL");
        }
        con.close();
    }
}
```

# JDBC ( Fetch Data)

```
1. Class.forName(" ... driver class Name .... ");
2. Connection con = DriverManager.getConnection("url","username","pswd");
3. PreparedStatement ps = con.prepareStatement("select * from register");
4. Resultset rs = ps.executeQuery();
5. While (rs.next())
   {
       String name1 = rs.getString("name");
       String email1 = rs.getString("email");
       -
       -
       -
       // print
   }

6. con.close();
```

# JDBC (FETCH)

| name | email | password | gender | city |
|---|---|---|---|---|
| A | a@gmail.com | 123 | M | delhi |
| B | b@--- | 456 | F | pune |
| C | c@--- | 678 | M | chd. |

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class SelectDemo
{
    psvm(—) throws Exception
    {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbc_db","root","root");
        PrepareStatement ps = con.prepareStatement("select * from register");
        Resultset rs = ps.executeQuery();
        While(rs.next())
        {
            SOP(rs.getString("name"));
            SOP(rs.getString("email"));
            SOP(rs.getString("password"));
            SOP(rs.getString("gender"));
            SOP(rs.getString("city"));
        }

        con.close();
    }
}
```

# JDBC [Delete]

```
public class DeleteDemo
{
    psvm ( — ) throws Exception
    {
      String email1 = "abc@gmail.com";
      class.forName ("com.mysql.cj.jdbc.Driver");
      Connection con = DriverManager.getConnection ("jdbc:mysql://localhost:3306/jdbc_db","root","root");
      PreparedStatement ps = con.prepareStatement ("delete from register where email = ?");
      ps.setString (1, email1);
      int Count = ps.executeUpdate ();
      if (Count > 0)
      {
        SOP ("SUCCESS");
      }
      else
      {
        SOP ("FAIL");
      }
      con.close();
    }
}
```